Understanding Contextual Embeddings

A first-principles, exploratory journey from static word vectors to dynamic, negotiated meaning — now with runnable code

---

Let's build this from the ground up. The goal isn't a definition you memorize, it's a picture you can see in your head whenever someone says “contextual embedding.” So we'll go slowly, and we'll let each idea break before we fix it — because the breaking is where the understanding lives. Alongside the prose, each part now includes a short, runnable code example so you can see the idea, not just read about it.

Part 1: The Pre-Contextual Era — Giving Words a Fixed Address in Space

Before we can appreciate what contextual embeddings do, we need to sit with the problem they were built to solve. That means going back to a genuinely clever idea: static word embeddings, like Word2Vec and GloVe.

The core insight these systems had was simple but profound: **a word's meaning can be approximated by the company it keeps.** This is the distributional hypothesis — “you shall know a word by the company it keeps.” If “dog” and “puppy” tend to appear near “bark,” “leash,” and “vet,” while “car” and “truck” appear near “engine,” “drive,” and “road,” then a machine can learn something about meaning just by watching which words co-occur across millions of sentences — without any human ever defining a “dog.”

So here is what these systems did, conceptually: they gave every word in the vocabulary a single, fixed point in a high-dimensional space — a location in a vast, invisible galaxy, perhaps 300 dimensions instead of the 3 we're used to. Words that behaved similarly in text ended up parked near each other. “King” and “queen” would be close. “Happy” and “joyful” would be close. Astonishingly, the *directions* between points captured relationships — the famous example being that “king” minus “man” plus “woman” lands very close to “queen.” The geometry of the space was doing something that looked like reasoning.

This was a genuine breakthrough. For the first time, “meaning” had shape, rather than words being arbitrary, unrelated symbols.

But here is the crack in the foundation, and it's worth sitting with it carefully.

Every word got exactly **one** vector. One fixed address. Permanently.

Think about what that means for the word “bank.” In a static embedding system, “bank” has to live at a single point in that 300-dimensional galaxy — for its entire existence, across every sentence it will ever appear in. But “bank” doesn't have one meaning. It has at least two wildly different ones:

> *“I sat by the river **bank** and watched the water flow.”*
>
> *“I deposited my paycheck at the investment **bank**.”*

These are almost entirely unrelated concepts — one about geography and nature, the other about finance and institutions. A human reads these two sentences and instantly assigns “bank” two different meanings without even noticing the ambiguity. The static embedding system has no such luxury. It was trained by averaging over *every context* the word ever appeared in — rivers, money, maybe billiards (“bank shot”), blood banks, memory banks. All of that gets mathematically blended into one static, compromise vector.

Picture it almost literally: imagine describing a person using only one photograph, but that photograph is actually a long-exposure shot where the shutter stayed open while the person did fifty completely different things — some days a swimmer, some days a banker, some days playing pool. The resulting photo isn't clearly any one of those things. It's a blur, an average, a ghost of many identities superimposed. That is the static vector for “bank”: the geometric mean of “river bank” and “investment bank” and every other bank, mashed into one point that fits none of them precisely.

The same happens with “fly.” Is it the verb, “to fly a kite,” living near “soar,” “pilot,” “airplane”? Or the noun, the buzzing insect, living near “mosquito,” “bug,” “swat”? A static embedding can't decide — it doesn't even know these are different *senses* of the word. It sees the string “f-l-y” and assigns one vector, forever, regardless of what's happening around it in any particular sentence.

Why is this mathematically limiting, precisely? A single point in space can only be in one place. It cannot simultaneously sit close to “river,” “water,” “shore” *and* close to “money,” “finance,” “loan.” Trying to be close to both clusters forces the point into an awkward middle territory that is neither riverbank nor financial institution — correctly wrong about everything. Any downstream task — translation, question answering, sentiment analysis — inherits that blur. The machine reads with permanent short-sightedness: it can tell you what a word *usually* means, on average, but never what it means right here, right now.

This was the wall the field hit. And it's the wall that had to be broken for language models to get any deeper.

Code: proving a static embedding really is one fixed vector, no matter the sentence

This uses Gensim's pretrained Word2Vec vectors. Notice: the vector for “bank” is retrieved by the word alone — the function doesn't even accept a sentence as input. It literally cannot know which sense you mean.

```
import gensim.downloader as api

# Load pretrained static word vectors (300-dim GloVe)
```

```
model = api.load("glove-wiki-gigaword-300")

# The SAME vector is returned regardless of context --
# the API doesn't even take a sentence as an argument.
vec_bank = model["bank"]
print("Shape of 'bank' vector:", vec_bank.shape)   # (300,)

# Prove it's identical every time, context or not
sentence1_bank = model["bank"]   # "I sat by the river bank"
sentence2_bank = model["bank"]   # "I deposited money at the bank"

import numpy as np
print("Are the two vectors identical?",
      np.array_equal(sentence1_bank, sentence2_bank))   # True, always

# See the consequence: 'bank' is dragged toward BOTH neighborhoods
print("similarity(bank, river):", model.similarity("bank", "river"))
print("similarity(bank, money):", model.similarity("bank", "money"))
# Both come out moderately positive -- a compromise, not a clear match
# to either sense.
```

Walking through it: we load GloVe vectors that were pretrained once, on a fixed corpus, and never touched again at inference time. When we call `model["bank"]`, notice the function signature — it takes only the string “bank,” nothing about the sentence it came from. That's the whole diagnosis in one line: there is no parameter for context, so there is no mechanism by which context could possibly matter. The `np.array_equal` check makes this undeniable — two calls representing two totally different sentences return bit-for-bit the same array. The similarity scores at the bottom show the compromise directly: “bank” isn't strongly tied to “river”, nor strongly tied to “money” — it's moderately similar to both, because its single vector had to statistically average over every sense it was ever used in during training.

Part 2: The Paradigm Shift — From “What Is This Word?” to “What Is This Word, Given Everything Around It?”

How do humans actually resolve this ambiguity? Introspect for a moment. When you read “bank,” you aren't consciously flipping through a dictionary and picking the right entry through deliberate reasoning. It happens instantly, almost pre-consciously. Your brain uses the surrounding words as *evidence*. “River” appeared three words earlier — a strong vote for the geographic sense. “Deposited,” “paycheck,” “loan” — votes for the financial sense. You aren't choosing a meaning in isolation; you're letting neighboring words *cast votes*, and the meaning crystallizes as a weighted consensus of its surroundings.

This is the intuition that had to be translated into mathematics: **a word's representation should not be a fixed property of the word in isolation. It should be a function of the word and its context, computed fresh every single time.**

This is a genuinely different philosophical stance about what “meaning” even is. In the static world, meaning was a *noun* — a fixed thing you looked up, like a dictionary entry. In the contextual world, meaning becomes a *verb* — something that happens, computed on the fly, existing only once you know the sentence it's embedded in. The word “bank” doesn't have a meaning sitting in storage waiting to be retrieved. It has a *process* that generates a meaning, live, in response to its neighbors.

This shift — from lookup table to computation — is really the entire conceptual leap. Everything else, including the architecture we discuss next, is engineering in service of making that computation possible and efficient.

Code: a toy “voting” function — the intuition before the math

*Before we touch real attention math, here's a deliberately simplified, hand-written stand-in for “neighboring words cast votes.” It's not how Transformers really compute things (no learned weights yet) — it's just to make the *idea* of context-as-a-function tangible.*

```
# A toy, hand-crafted stand-in for "context casts votes" --
# NOT real attention math, just the intuition made concrete.

geo_words   = {"river", "shore", "water", "stream"}
fin_words   = {"deposited", "paycheck", "loan", "investment", "money"}

def toy_contextual_sense(sentence_tokens, target="bank"):
    votes_geo = sum(1 for w in sentence_tokens if w in geo_words)
    votes_fin = sum(1 for w in sentence_tokens if w in fin_words)
    total = votes_geo + votes_fin
    if total == 0:
        return {"geographic": 0.5, "financial": 0.5}   # no evidence -> stay ambiguous
    return {"geographic": votes_geo / total,
            "financial":  votes_fin / total}

s1 = ["i", "sat", "by", "the", "river", "bank"]
s2 = ["i", "deposited", "my", "paycheck", "at", "the", "bank"]

print("Sense distribution, sentence 1:", toy_contextual_sense(s1))
# -> {'geographic': 1.0, 'financial': 0.0}
print("Sense distribution, sentence 2:", toy_contextual_sense(s2))
# -> {'geographic': 0.0, 'financial': 1.0}

# The SAME word "bank" now resolves to different senses depending
# purely on which words happen to surround it. That's the whole
# philosophical shift -- meaning as a computed function of context,
# not a fixed lookup. Real attention (Part 3) does this with learned,
# continuous weights instead of hand-written word lists.
```

Walking through it: `toy_contextual_sense` takes the whole tokenized sentence, not just the target word — that's the key structural change from Part 1's code. It counts how many surrounding tokens belong to a hand-picked “geographic” bag of words versus a “financial” bag, then normalizes those counts into a probability-like split. When “river” is present, geographic gets 100% of the “vote”; when “deposited” and “paycheck” are present, financial gets 100%. This is deliberately crude — real models don't use hand-written word lists, they use learned, continuous, graded weights — but the

function signature itself already embodies the paradigm shift: the output depends on the whole sentence *(sentence_tokens)*, not on the target word in isolation. That's the one structural fact worth remembering from this snippet.

Part 3: The Inner Workings — How Words “Negotiate” Meaning with Each Other

Now the heart of the mechanism, built up in intuitive layers rather than dropped as equations.

The starting point: everyone begins with a rough, static guess.

When a sentence enters a Transformer-based model (like the one underlying BERT), each word (technically each “token”) starts life with an initial embedding — not unlike the static embeddings discussed earlier. Think of this as each word walking into a room holding a rough, generic definition card. “Bank” walks in holding a card that says something vague and averaged: “??? — possibly geographic, possibly financial, possibly other.”

Then something remarkable happens: the words start talking to each other.

This is the essence of the *self-attention mechanism*. Picture everyone in the sentence standing in a circle, and before anyone commits to what they mean, each word looks around at every other word and asks: “how relevant are you to figuring out what I mean?”

Concretely, here is the negotiation process, described intuitively. Every word generates three things about itself — three roles it plays in the conversation. It creates a **query**: the word raising its hand, broadcasting “here's the kind of information I'm looking for to clarify myself.” It creates a **key**: a name tag broadcasting “here's the kind of information I can offer to others.” And it creates a **value**: the actual substance it will contribute if someone finds it relevant — “here's what I'll actually give you if you're interested.”

So when “bank” is figuring out what it means in “I sat by the river bank,” it sends out its query — asking the room, “does anyone have geographic or financial evidence for me?” Every other word — “I,” “sat,” “by,” “the,” “river” — responds by comparing “bank's” query against their own keys. “River” has a key that resonates very strongly (mathematically, the dot product between “bank's” query vector and “river's” key vector is large). “Sat” and “by” resonate more weakly. “I” resonates barely at all.

This resonance score becomes an **attention weight** — essentially a percentage, where all weights across the sentence sum to 100%. “River” might get 60% of bank's attention, “sat” might get 15%, and so on. Then — the crucial step — “bank” updates its own representation by taking a weighted blend of everyone's *value* vectors, weighted by those percentages. Since “river” got the highest weight, its value vector dominates the update. “Bank's” vector shifts, pulling it geometrically closer to the river-and-shore region of the meaning-space, and away from the finance region.

In the other sentence — “I deposited my paycheck at the investment bank” — the exact same word “bank” runs the exact same process, but this time “deposited,” “paycheck,” and “investment” have high-resonance keys. “Bank's” query lights up differently, the attention weights land differently, and the resulting blended vector gets pulled toward the financial cluster instead.

This is the “absorption” made precise.

The word literally recomputes its own vector by absorbing a weighted mixture of the *value* vectors of its neighbors, where the weights reflect how relevant each neighbor judges itself to be (via query-key resonance). It isn't just proximity in the sentence that matters — a word ten positions away that's semantically crucial (like “river”) can dominate the update, while a word right next door that's semantically empty (like “the”) can be almost entirely ignored. This is a real departure from older methods (like RNNs) that mostly cared about sequential, nearby context — attention lets any word talk directly to any other word, regardless of distance.

Critically, this doesn't happen just once. In a real Transformer, this negotiation runs through many stacked layers, sometimes dozens. In the first layer, “bank” absorbs a fairly shallow, direct signal from its literal neighbors. That *updated* representation then feeds into a second round of attention, where “bank” (already partially disambiguated) negotiates again, this time with neighbors that have *also* been updated by their own first round. Several layers deep, the representations have absorbed increasingly abstract, increasingly contextualized information — not just “which nearby word is relevant” but genuinely rich compositional meaning, including syntactic role, discourse context, even something approximating world knowledge. Each layer is another round at the negotiating table, with everyone's positions slightly more refined than the round before.

One more piece worth holding onto: this whole process is *learned*, not hand-designed. Nobody told the model “pay attention to ‘river’ when you see ‘bank.’” During training — typically on a task like predicting masked-out words across billions of sentences — the model gradually discovers, purely through trial and error via gradient descent, that attending to “river” helps it correctly predict what comes next or fill in blanks correctly. The query, key, and value transformations are just learned weight matrices, tuned until this useful attention behavior emerges as a side effect of getting really good at the underlying language task.

Code: real self-attention, from scratch, in plain NumPy

This is the actual mathematical mechanism — simplified to one attention head, with random (untrained) weight matrices just to show the mechanics: query/key resonance -> softmax -> weighted blend of values.

```
import numpy as np

np.random.seed(0)
d = 8              # embedding dimension (tiny, just for illustration)
tokens = ["i", "sat", "by", "the", "river", "bank"]
n = len(tokens)

# Step 0: start with rough, random initial embeddings (stand-in for
```

```
# what a real model would start with -- e.g. static-ish embeddings)
X = np.random.randn(n, d)

# Step 1: learned projection matrices (random here, but in a real
# model these are trained via backprop over billions of sentences)
W_q = np.random.randn(d, d) * 0.1
W_k = np.random.randn(d, d) * 0.1
W_v = np.random.randn(d, d) * 0.1

Q = X @ W_q     # every word's "query"
K = X @ W_k     # every word's "key"
V = X @ W_v     # every word's "value"

# Step 2: how strongly does "bank" (last token) resonate with
# every other word? -> dot product of bank's query with every key
bank_idx = tokens.index("bank")
scores = Q[bank_idx] @ K.T / np.sqrt(d)     # scaled dot-product

# Step 3: turn resonance scores into attention weights (sum to 1)
def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

attention_weights = softmax(scores)

for tok, w in zip(tokens, attention_weights):
    print(f"{tok:>6s}: {w:.3f}")

# Step 4: "bank" absorbs a weighted blend of everyone's VALUE vectors
bank_new_vector = attention_weights @ V
print("\nUpdated 'bank' vector (first 4 dims):", bank_new_vector[:4])

# With trained (not random) weights, you would see "river" pull the
# highest attention weight -- and bank_new_vector would sit measurably
# closer to geographic-sense words than to financial-sense words.
```

Walking through it, step by step: Step 0 gives every word a random starting vector `X` — a stand-in for whatever embedding the word walked in holding. Step 1 creates three learned projections of every word: `Q` (queries, “what am I looking for”), `K` (keys, “what can I offer”), and `V` (values, “what I'll actually contribute”) — these are just the embeddings multiplied by three separate weight matrices, exactly as described in the prose. Step 2 is the resonance check: we take *only* “bank's” query row and dot it against every word's key row (`Q[bank_idx] @ K.T`), producing one raw compatibility score per neighbor — dividing by `sqrt(d)` is a standard scaling trick to keep these numbers from growing too large as dimensionality increases. Step 3 runs `softmax` on those scores, which is the mathematical operation that turns arbitrary numbers into a clean probability distribution summing to 1 — this is precisely the “attention weight as a percentage” idea from the prose. Step 4 is the absorption itself: `attention_weights @ V` multiplies each neighbor's value vector by how much attention it received and sums them up, producing “bank's” brand-new, context-infused vector. Because the weight matrices here are random and untrained, the numbers won't show anything meaningful yet — but wire this exact same computation into a model trained on billions of sentences, and “river” would

reliably earn the highest attention weight in this sentence, exactly as the prose describes.

Part 4: The High-Dimensional Picture — What the Landscape Actually Looks Like

Now let's visualize the payoff of all this machinery.

With a **static embedding**, plotting the word “bank” gives you exactly *one dot*, sitting somewhere in that 300-dimensional galaxy, forever fixed, regardless of how many sentences you feed it. Compressed down to two dimensions for visualization (using something like PCA or t-SNE, common ways to squash high-dimensional data into a viewable plot), “bank” appears as a single, lonely point — probably sitting in awkward no-man's-land between the “finance” neighborhood and the “geography” neighborhood, since it had to compromise between both.

Now imagine feeding **fifty different sentences** containing “bank” into a contextual model, and plotting the resulting vector from each one. What you'd see is startlingly different: not a single dot, but a *cloud* — or more precisely, an archipelago of distinct clusters.

Sentences about riverbanks, shorelines, and geography would cluster tightly together in one region of the space — near vectors for “shore,” “riverbed,” “embankment.” Sentences about financial institutions would form a completely separate cluster, floating near “loan,” “credit,” “vault,” “teller.” And if your fifty sentences included rarer senses — a “blood bank,” a “memory bank” in a computer, a “bank shot” in pool — you might see smaller satellite clusters branching off, each capturing a distinct sense, positioned according to how semantically related it is to the others (a “data bank” might sit somewhat nearer the “memory bank” cluster than the “riverbank” cluster, since both involve storage).

What's genuinely beautiful about this picture is that the *distance between clusters* is itself meaningful. The riverbank cluster and the financial-bank cluster are usually far apart, reflecting how semantically unrelated those senses are. But within the financial-bank cluster, you'd likely see finer-grained structure too — sentences about “bank” as a large multinational institution might sit slightly apart from sentences about “bank” as a small local credit union, even though both are clearly in “financial” territory. The geometry isn't just binary (river vs. finance); it's continuous and graded, capturing subtle shifts in nuance a static system couldn't begin to represent.

Here's a detail that often deepens the intuition further: even the *exact same word in the exact same sense* will produce slightly different vectors across different sentences, because context isn't just about disambiguating word *sense* — it's about capturing everything relevant happening in that sentence: tone, emphasis, syntactic role, what's being discussed nearby. Two sentences both clearly about “investment bank” will still produce two nearby-but-not-identical points, because a contextual embedding is a snapshot of “this word, doing this specific job, in this specific sentence” — not just “this word, in this general category.”

So if a static embedding gives you a *city* on a map — one fixed location representing an average of everywhere that word has ever been used — a contextual embedding gives you a *trajectory*, a live GPS signal that moves depending on where the word currently finds itself, snapping into the neighborhood that actually matches its role in this particular sentence, at this particular moment, before dissolving and being recomputed fresh the very next time that word appears somewhere else.

That shift — from a fixed address to a live, negotiated, ever-recomputed position — is the entire conceptual heart of contextual embeddings. Everything else you'll encounter (positional encodings, multi-head attention, layer normalization, and so on) is refinement and machinery in service of that one central idea: meaning isn't stored, it's **computed**, fresh, every time, from the company a word keeps.

Code: proving it with a real pretrained model (BERT via HuggingFace)

This uses an actual trained Transformer. Unlike the Gensim example in Part 1, note the function signature: it takes a whole SENTENCE, extracts the vector for 'bank' from its final hidden layer, and the vector genuinely differs by context. We confirm it numerically with cosine similarity.

```
from transformers import AutoTokenizer, AutoModel
import torch
import torch.nn.functional as F

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

def get_word_vector(sentence, word):
    inputs = tokenizer(sentence, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs)
    hidden_states = outputs.last_hidden_state[0]   # (seq_len, 768)

    # Find which subword token(s) correspond to our target word
    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
    idx = tokens.index(word)
    return hidden_states[idx]

s_geo = "I sat by the river bank and watched the water flow."
s_fin = "I deposited my paycheck at the investment bank."
s_geo2 = "The children played along the muddy bank near the stream."

vec_geo  = get_word_vector(s_geo, "bank")
vec_fin  = get_word_vector(s_fin, "bank")
vec_geo2 = get_word_vector(s_geo2, "bank")

def cos_sim(a, b):
    return F.cosine_similarity(a.unsqueeze(0), b.unsqueeze(0)).item()

print("river-sense vs financial-sense 'bank':", cos_sim(vec_geo, vec_fin))
print("river-sense vs another river-sense 'bank':", cos_sim(vec_geo, vec_geo2))

# Typical result:
#   river-sense vs financial-sense:  ~0.45-0.55   (noticeably different)
```

```
#   river-sense vs another river-sense: ~0.75-0.85 (much more similar)
#
# Compare this to Part 1's static embedding, where BOTH comparisons
# would have used the exact same, single vector -- similarity of 1.0
# to itself, with no way to distinguish sense at all.
```

Walking through it: `get_word_vector` is the important function to study here — compare its signature to Part 1's. It takes a full `sentence` plus a target `word`, tokenizes the whole sentence, and runs it through BERT's entire stack of self-attention layers (that's what `model(**inputs)` is doing under the hood — the exact negotiation process from Part 3, repeated across roughly 12 layers). We pull `last_hidden_state`, which holds one final, fully-contextualized vector per token, then locate the specific token matching our target word and extract just its vector. We deliberately call this function three times with “bank” embedded in three different sentences. Then `cos_sim` measures how aligned two vectors are in direction (1.0 = identical direction, 0 = unrelated, negative = opposite) — it's the standard way to ask “how similar are these two meanings, geometrically?” The result is the empirical payoff of the entire journey: the two river-sense vectors land close together (high cosine similarity), while the river-sense and finance-sense vectors land measurably farther apart — even though it's literally the same three-letter word in all three sentences. That numeric gap is Part 4's “archipelago of clusters” made concrete in a single comparison.

A note on running these yourself: the Gensim example needs `pip install gensim` and downloads ~65MB of vectors on first run; the NumPy example needs nothing but NumPy; the BERT example needs `pip install transformers torch` and downloads the bert-base-uncased weights (~440MB) on first run.